



Prolećni semestar 2025/2026

PREDMET: CS230

PROJEKTNi ZADATAK

Event-Based pretplatnički sistem za sportske rezultate

STUDENT

Pavle Zlatanović 6406
Informacione tehnologije

ASISTENT

Anđela Grujić

Niš, 05.06.2026.

Sadržaj

1. Uvod i teorijska postavka
 - 1.1 Distribuirani sistemi
 - 1.2 Publish-Subscribe paradigma
 - 1.3 Event-Based pretplatnički sistemi
 - 1.4 TCP i socket programiranje
 - 1.5 Threading u Python-u
2. Studija slučaja – implementacija sistema
 - 2.1 Opis sistema i scenario
 - 2.2 Arhitektura sistema
 - 2.3 Implementacija komponenti
 - 2.3.1 Publisher
 - 2.3.2 Server
 - 2.3.3 Client
 - 2.3.4 Database
 - 2.3.5 Middleware
 - 2.4 Tok podataka kroz sistem
 - 2.5 Rad sa SQLite bazom i JSON datotekama
 - 2.6 Testiranje sistema
3. Zaključak
4. Literatura

1. Uvod i teorijska postavka

Danas se veliki broj aplikacija zasniva na distribuiranim sistemima. Umesto da sve radi na jednom računaru, posao se raspoređuje na više povezanih komponenti. Korisnik uglavnom ne vidi tu složenost već koristi sistem kao jednu celinu.

U ovom radu prikazan je event-based pretplatnički sistem za sportske rezultate. Sistem omogućava unos rezultata utakmica, njihovo čuvanje i automatsko obaveštavanje korisnika koji prate određene timove.

Projekat je razvijen u Python programskom jeziku korišćenjem TCP socket komunikacije. Rezultati se čuvaju u SQLite bazi podataka i JSON datoteci.

1.1 Distribuirani sistemi

Distribuirani sistem predstavlja skup računara ili procesa koji međusobno sarađuju preko mreže. Prednosti ovakvih sistema su skalabilnost, dostupnost i mogućnost rada većeg broja

korisnika istovremeno.

Primeri distribuiranih sistema mogu se pronaći u svakodnevnom životu. Internet pretraživači, društvene mreže i servisi za strimovanje koriste veliki broj servera koji rade zajedno.

U ovom projektu distribuirana priroda sistema se vidi kroz odvojene procese servera, publisher-a i klijenata koji međusobno komuniciraju preko mreže.

Distribuirani sistem predstavlja skup računara ili procesa koji međusobno sarađuju preko mreže. Prednosti ovakvih sistema su skalabilnost, dostupnost i mogućnost rada većeg broja korisnika istovremeno.

Primeri distribuiranih sistema mogu se pronaći u svakodnevnom životu. Internet pretraživači, društvene mreže i servisi za strimovanje koriste veliki broj servera koji rade zajedno.

U ovom projektu distribuirana priroda sistema se vidi kroz odvojene procese servera, publisher-a i klijenata koji međusobno komuniciraju preko mreže.

1.2 Publish-Subscribe paradigma

Publish-Subscribe model omogućava komunikaciju između komponenti bez direktne povezanosti. Publisher šalje informacije, subscriber prima informacije, dok broker ili server prosleđuje događaje.

Glavna prednost ovog modela je jednostavno proširenje sistema. Novi korisnici mogu da se povežu bez promene postojeće logike.

U ovom projektu sportski rezultati predstavljaju događaje koji se šalju zainteresovanim korisnicima.

Publish-Subscribe model omogućava komunikaciju između komponenti bez direktne povezanosti. Publisher šalje informacije, subscriber prima informacije, dok broker ili server prosleđuje događaje.

Glavna prednost ovog modela je jednostavno proširenje sistema. Novi korisnici mogu da se povežu bez promene postojeće logike.

U ovom projektu sportski rezultati predstavljaju događaje koji se šalju zainteresovanim korisnicima.

1.3 Event-Based pretplatnički sistemi

Event-Based sistemi zasnivaju se na događajima. Kada se dogodi promena, kreira se događaj koji se šalje svim odgovarajućim pretplatnicima.

U projektu novi rezultat utakmice predstavlja događaj. Server prima događaj, obrađuje ga i prosleđuje svim korisnicima koji prate odgovarajući tim ili svim korisnicima koji su izabrali opciju praćenja svih timova.

Ovakav pristup je jednostavan za implementaciju i veoma pogodan za sisteme obaveštenja u realnom vremenu.

Event-Based sistemi zasnivaju se na događajima. Kada se dogodi promena, kreira se događaj koji se šalje svim odgovarajućim pretplatnicima.

U projektu novi rezultat utakmice predstavlja događaj. Server prima događaj, obrađuje ga i prosleđuje svim korisnicima koji prate odgovarajući tim ili svim korisnicima koji su izabrali opciju praćenja svih timova.

Ovakav pristup je jednostavan za implementaciju i veoma pogodan za sisteme obaveštenja u realnom vremenu.

1.4 TCP i socket programiranje

TCP je protokol koji omogućava pouzdanu komunikaciju između procesa. Za razliku od UDP protokola, TCP garantuje da će podaci stići ispravnim redosledom.

Python biblioteka socket korišćena je za implementaciju komunikacije između svih komponenti sistema. Server osluškuje konekcije na portu 5000, dok se publisher i klijenti povezuju na njega.

Korišćenje socket programiranja omogućilo je razvoj jednostavnog distribuiranog sistema bez dodatnih biblioteka.

TCP je protokol koji omogućava pouzdanu komunikaciju između procesa. Za razliku od UDP protokola, TCP garantuje da će podaci stići ispravnim redosledom.

Python biblioteka socket korišćena je za implementaciju komunikacije između svih komponenti sistema. Server osluškuje konekcije na portu 5000, dok se publisher i klijenti povezuju na njega.

Korišćenje socket programiranja omogućilo je razvoj jednostavnog distribuiranog sistema bez dodatnih biblioteka.

1.5 Threading u Python-u

Server mora istovremeno da radi sa više klijenata. Zbog toga je korišćen modul threading.

Za svakog povezanog klijenta kreira se poseban thread. Na taj način jedan korisnik ne blokira ostale korisnike.

Threading omogućava da server prihvata nove konekcije dok istovremeno obrađuje postojeće korisnike i nove sportske rezultate.

Server mora istovremeno da radi sa više klijenata. Zbog toga je korišćen modul threading.

Za svakog povezanog klijenta kreira se poseban thread. Na taj način jedan korisnik ne blokira ostale korisnike.

Threading omogućava da server prihvata nove konekcije dok istovremeno obrađuje postojeće korisnike i nove sportske rezultate.

2.1 Opis sistema i scenario

Sistem služi za distribuciju sportskih rezultata u realnom vremenu. Administrator koristi publisher aplikaciju za unos rezultata utakmica.

Korisnici se preko client aplikacije pretplaćuju na određeni tim. Kada se unese novi rezultat, server proverava pretplate i šalje obaveštenje odgovarajućim korisnicima.

Primer scenarija:

Korisnik A prati Crvenu zvezdu.

Korisnik B prati Partizan.

Korisnik C prati sve timove.

Ako se unese rezultat utakmice Crvena zvezda – Radnički, obaveštenje dobijaju korisnici A i C.

Sistem služi za distribuciju sportskih rezultata u realnom vremenu. Administrator koristi publisher aplikaciju za unos rezultata utakmica.

Korisnici se preko client aplikacije pretplaćuju na određeni tim. Kada se unese novi rezultat, server proverava pretplate i šalje obaveštenje odgovarajućim korisnicima.

Primer scenarija:

Korisnik A prati Crvenu zvezdu.

Korisnik B prati Partizan.

Korisnik C prati sve timove.

Ako se unese rezultat utakmice Crvena zvezda – Radnički, obaveštenje dobijaju korisnici A i C.

Sistem služi za distribuciju sportskih rezultata u realnom vremenu. Administrator koristi publisher aplikaciju za unos rezultata utakmica.

Korisnici se preko client aplikacije pretplaćuju na određeni tim. Kada se unese novi rezultat, server proverava pretplate i šalje obaveštenje odgovarajućim korisnicima.

Primer scenarija:

Korisnik A prati Crvenu zvezdu.

Korisnik B prati Partizan.

Korisnik C prati sve timove.

Ako se unese rezultat utakmice Crvena zvezda – Radnički, obaveštenje dobijaju korisnici A i C.

2.2 Arhitektura sistema

Sistem se sastoji od pet glavnih komponenti: server.py, client.py, publisher.py, database.py i middleware.py.

Server predstavlja centralnu komponentu sistema. Publisher unosi rezultate, client prima obaveštenja, database čuva podatke, a middleware vodi log aktivnosti.

Sve komponente komuniciraju preko TCP konekcije.

Sistem se sastoji od pet glavnih komponenti: server.py, client.py, publisher.py, database.py i middleware.py.

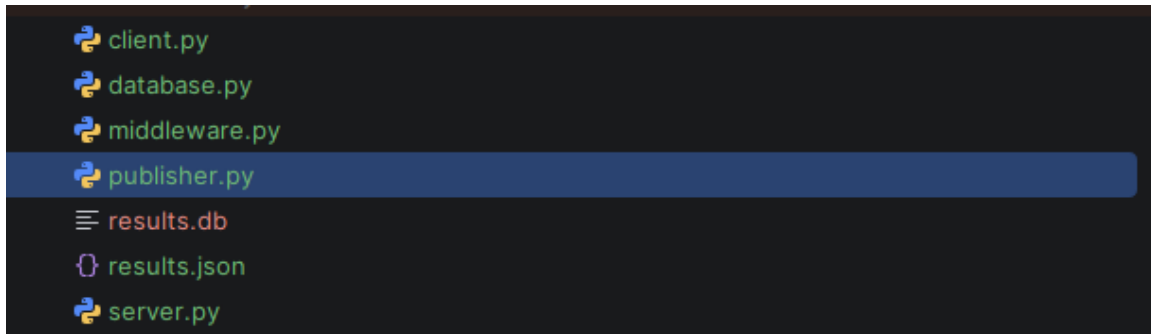
Server predstavlja centralnu komponentu sistema. Publisher unosi rezultate, client prima obaveštenja, database čuva podatke, a middleware vodi log aktivnosti.

Sve komponente komuniciraju preko TCP konekcije.

Sistem se sastoji od pet glavnih komponenti: server.py, client.py, publisher.py, database.py i middleware.py.

Server predstavlja centralnu komponentu sistema. Publisher unosi rezultate, client prima obaveštenja, database čuva podatke, a middleware vodi log aktivnosti.

Sve komponente komuniciraju preko TCP konekcije.



2.3.1 Publisher

Publisher predstavlja administratorsku aplikaciju. Nakon pokretanja prikazuje meni za dodavanje rezultata i pregled postojećih rezultata.

Prilikom unosa rezultata kreira se poruka u formatu RESULT;domaci;gost;rezultat koja se šalje serveru.

Publisher takođe može da pročita sve rezultate iz baze podataka.

Publisher predstavlja administratorsku aplikaciju. Nakon pokretanja prikazuje meni za dodavanje rezultata i pregled postojećih rezultata.

Prilikom unosa rezultata kreira se poruka u formatu RESULT;domaci;gost;rezultat koja se šalje serveru.

Publisher takođe može da pročita sve rezultate iz baze podataka.

Publisher predstavlja administratorsku aplikaciju. Nakon pokretanja prikazuje meni za dodavanje rezultata i pregled postojećih rezultata.

Prilikom unosa rezultata kreira se poruka u formatu RESULT;domaci;gost;rezultat koja se šalje serveru.

Publisher takođe može da pročita sve rezultate iz baze podataka.

```
1 - Dodaj rezultat
2 - Prikazi rezultate
3 - Izlaz
Izbor:
```

2.3.2 Server

Server predstavlja centralni broker sistema. Njegov zadatak je prihvatanje konekcija, obrada pretplata i distribucija događaja.

Server čuva rečnik subscribers u kojem se nalaze informacije o korisnicima i timovima koje prate.

Kada primi novi rezultat, server ga čuva u bazi i JSON datoteci, a zatim obaveštava pretplaćene korisnike.

Server predstavlja centralni broker sistema. Njegov zadatak je prihvatanje konekcija, obrada pretplata i distribucija događaja.

Server čuva rečnik subscribers u kojem se nalaze informacije o korisnicima i timovima koje prate.

Kada primi novi rezultat, server ga čuva u bazi i JSON datoteci, a zatim obaveštava pretplaćene korisnike.

Server predstavlja centralni broker sistema. Njegov zadatak je prihvatanje konekcija, obrada pretplata i distribucija događaja.

Server čuva rečnik subscribers u kojem se nalaze informacije o korisnicima i timovima koje prate.

Kada primi novi rezultat, server ga čuva u bazi i JSON datoteci, a zatim obaveštava pretplaćene korisnike.

```
[02:10:12] Server pokrenut
```

2.3.3 Client

Client omogućava korisniku da prati rezultate određenog tima. Nakon povezivanja korisnik unosi naziv tima koji želi da prati.

Poseban thread stalno osluškuje dolazne poruke sa servera. Kada stigne novi rezultat, on se prikazuje korisniku u konzoli.

Client omogućava korisniku da prati rezultate određenog tima. Nakon povezivanja korisnik unosi naziv tima koji želi da prati.

Poseban thread stalno osluškuje dolazne poruke sa servera. Kada stigne novi rezultat, on se prikazuje korisniku u konzoli.

Client omogućava korisniku da prati rezultate određenog tima. Nakon povezivanja korisnik unosi naziv tima koji želi da prati.

Poseban thread stalno osluškuje dolazne poruke sa servera. Kada stigne novi rezultat, on se prikazuje korisniku u konzoli.

```
Koji tim pratite? Zvezda
Pretplaceni ste na tim: Zvezda
```

2.3.4 Database

Za čuvanje podataka koristi se SQLite baza podataka. Tabela results sadrži naziv domaćeg tima, gostujućeg tima i rezultat.

SQLite je izabran zato što je jednostavan za korišćenje i ne zahteva instalaciju dodatnog servera baze podataka.

Rezultati ostaju sačuvani i nakon gašenja aplikacije.

Za čuvanje podataka koristi se SQLite baza podataka. Tabela results sadrži naziv domaćeg tima, gostujućeg tima i rezultat.

SQLite je izabran zato što je jednostavan za korišćenje i ne zahteva instalaciju dodatnog servera baze podataka.

Rezultati ostaju sačuvani i nakon gašenja aplikacije.

Za čuvanje podataka koristi se SQLite baza podataka. Tabela results sadrži naziv domaćeg tima, gostujućeg tima i rezultat.

SQLite je izabran zato što je jednostavan za korišćenje i ne zahteva instalaciju dodatnog servera baze podataka.

Rezultati ostaju sačuvani i nakon gašenja aplikacije.

2.3.5 Middleware

Middleware služi za logovanje aktivnosti sistema. Svaka važna akcija dobija vremensku oznaku.

Logovi olakšavaju praćenje rada sistema i pronalaženje eventualnih grešaka tokom testiranja.

Middleware služi za logovanje aktivnosti sistema. Svaka važna akcija dobija vremensku oznaku.

Logovi olakšavaju praćenje rada sistema i pronalaženje eventualnih grešaka tokom testiranja.

Middleware služi za logovanje aktivnosti sistema. Svaka važna akcija dobija vremensku oznaku.

Logovi olakšavaju praćenje rada sistema i pronalaženje eventualnih grešaka tokom testiranja.

2.4 Tok podataka kroz sistem

Administrator unosi rezultat preko publisher aplikacije. Publisher šalje poruku serveru.

Server prima poruku, čuva rezultat u bazi i JSON datoteci. Nakon toga proverava listu pretplatnika.

Svi korisnici koji prate odgovarajući tim dobijaju obaveštenje o novom rezultatu. Na taj način ostvarena je event-based komunikacija.

Administrator unosi rezultat preko publisher aplikacije. Publisher šalje poruku serveru.

Server prima poruku, čuva rezultat u bazi i JSON datoteci. Nakon toga proverava listu pretplatnika.

Svi korisnici koji prate odgovarajući tim dobijaju obaveštenje o novom rezultatu. Na taj način ostvarena je event-based komunikacija.

Administrator unosi rezultat preko publisher aplikacije. Publisher šalje poruku serveru.

Server prima poruku, čuva rezultat u bazi i JSON datoteci. Nakon toga proverava listu pretplatnika.

Svi korisnici koji prate odgovarajući tim dobijaju obaveštenje o novom rezultatu. Na taj način ostvarena je event-based komunikacija.

```
Izbor: 1
Domaci tim: Zvezda
Gostujuci tim: Partizan
Rezultat: 5:0
```

NOVI REZULTAT: Zvezda 5:0 Partizan

2.5 Rad sa SQLite bazom i JSON datotekama

Prilikom svakog unosa rezultata vrši se dvostruko čuvanje podataka.

Prvo se rezultat upisuje u SQLite bazu podataka. Nakon toga isti podaci se upisuju u JSON datoteku results.json.

Ovakav pristup omogućava veću pouzdanost i lakši pregled podataka.

Prilikom svakog unosa rezultata vrši se dvostruko čuvanje podataka.

Prvo se rezultat upisuje u SQLite bazu podataka. Nakon toga isti podaci se upisuju u JSON datoteku results.json.

Ovakav pristup omogućava veću pouzdanost i lakši pregled podataka.

Prilikom svakog unosa rezultata vrši se dvostruko čuvanje podataka.

Prvo se rezultat upisuje u SQLite bazu podataka. Nakon toga isti podaci se upisuju u JSON datoteku results.json.

Ovakav pristup omogućava veću pouzdanost i lakši pregled podataka.

```

1  [
2      {
3          "home_team": "Zvezda",
4          "away_team": "Partizan",
5          "score": "3:0"
6      },
7      {
8          "home_team": "Zvezda",
9          "away_team": "Partizan",
10         "score": "3:0"
11     },
12     {
13         "home_team": "Zvezda",
14         "away_team": "Bayern",
15         "score": "2:1"
16     },
17     {
18         "home_team": "Zvezda",
19         "away_team": "Partizan",
20         "score": "5:0"
21     }
22 ]

```

2.6 Testiranje sistema

Testiranje je izvršeno u PyCharm razvojnom okruženju. Istovremeno su pokrenuti server, publisher i više client aplikacija.

Provereno je da korisnici dobijaju samo rezultate timova koje prate. Takođe je potvrđeno da se podaci pravilno upisuju u bazu i JSON datoteku.

Sistem je uspešno radio sa više istovremenih klijenata bez prekida komunikacije.

Testiranje je izvršeno u PyCharm razvojnom okruženju. Istovremeno su pokrenuti server, publisher i više client aplikacija.

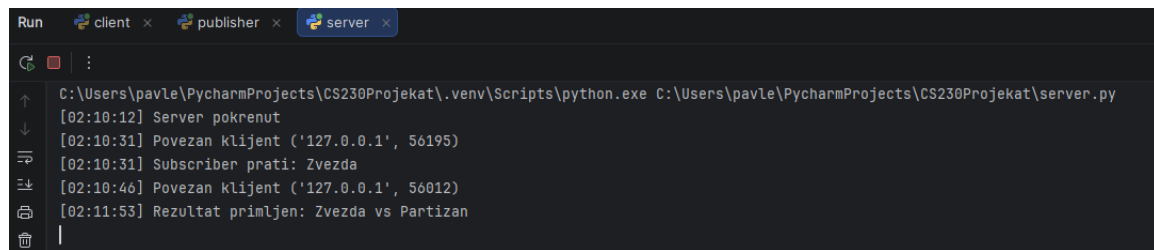
Provereno je da korisnici dobijaju samo rezultate timova koje prate. Takođe je potvrđeno da se podaci pravilno upisuju u bazu i JSON datoteku.

Sistem je uspešno radio sa više istovremenih klijenata bez prekida komunikacije.

Testiranje je izvršeno u PyCharm razvojnom okruženju. Istovremeno su pokrenuti server, publisher i više client aplikacija.

Provereno je da korisnici dobijaju samo rezultate timova koje prate. Takođe je potvrđeno da se podaci pravilno upisuju u bazu i JSON datoteku.

Sistem je uspešno radio sa više istovremenih klijenata bez prekida komunikacije.

The image shows a screenshot of the PyCharm Run console. At the top, there are three tabs: 'client', 'publisher', and 'server'. The 'server' tab is active. The console output shows the following logs:

```
C:\Users\pavle\PycharmProjects\CS230Projekat\.venv\Scripts\python.exe C:\Users\pavle\PycharmProjects\CS230Projekat\server.py
[02:10:12] Server pokrenut
[02:10:31] Povezan klijent ('127.0.0.1', 56195)
[02:10:31] Subscriber prati: Zvezda
[02:10:46] Povezan klijent ('127.0.0.1', 56012)
[02:11:53] Rezultat primljen: Zvezda vs Partizan
```

3. Zaključak

U okviru projektnog zadatka razvijen je event-based pretplatnički sistem za sportske rezultate. Projekat uspešno demonstrira rad distribuiranih sistema, Publish-Subscribe modela i TCP komunikacije.

Korišćenjem SQLite baze omogućeno je trajno čuvanje podataka, dok threading omogućava rad sa više korisnika istovremeno.

Sistem se može dodatno unaprediti dodavanjem grafičkog interfejsa, autentifikacije korisnika ili web verzije aplikacije.

U okviru projektnog zadatka razvijen je event-based pretplatnički sistem za sportske rezultate. Projekat uspešno demonstrira rad distribuiranih sistema, Publish-Subscribe modela i TCP komunikacije.

Korišćenjem SQLite baze omogućeno je trajno čuvanje podataka, dok threading omogućava rad sa više korisnika istovremeno.

Sistem se može dodatno unaprediti dodavanjem grafičkog interfejsa, autentifikacije korisnika ili web verzije aplikacije.

4. Literatura

- [1] Tanenbaum, Distributed Systems
- [2] Python Socket Documentation
- [3] Python Threading Documentation
- [4] SQLite Documentation
- [5] Publish-Subscribe Systems Survey

- [1] Tanenbaum, Distributed Systems
- [2] Python Socket Documentation
- [3] Python Threading Documentation
- [4] SQLite Documentation
- [5] Publish-Subscribe Systems Survey